

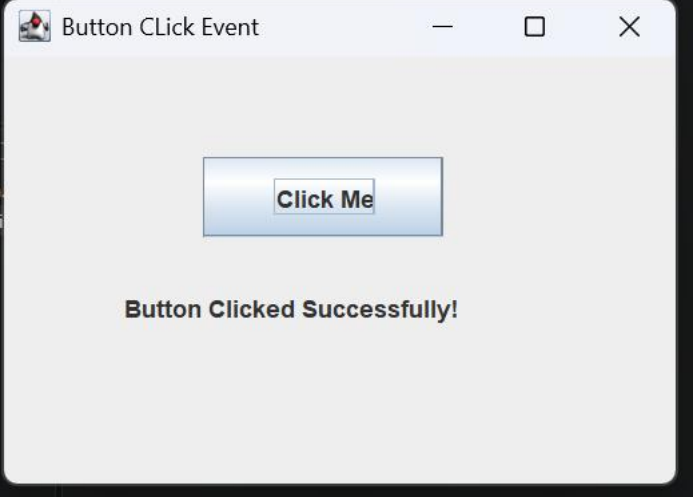


Symbiosis Institute of Technology, Pune

CSE- Academic Year 2026-27

Java Programming – Lab Batch 2025-29

Lab Assignment No.: 18	
Name of Student	Akshit Gawade
PRN No.	25070122029
Batch	2025-2029
Class	A1
Academic Year & Semester	Second Year Semester III
Date of Submission	
Title of Assignment:	Write a Java GUI program to demonstrate button click event handling.
Problem Statement & Learning Objective	Problem Statement Write a Java GUI program to demonstrate button click event handling. Learning Objectives To implement button click events using Java event handling mechanisms.
Theory	Event handling allows Java programs to respond to user actions such as button clicks, mouse movements, and keyboard input. Swing uses ActionListener to process button click events. Event-driven programming separates user interface design from event-processing logic, making applications interactive and user-friendly
Program Code	<pre>import java.awt.event.ActionEvent; import java.awt.event.ActionListener; import javax.swing.*; public class Main { public static void main (String[] args) { JFrame frame = new JFrame("Button Click Event"); frame.setSize(350, 250); frame.setLayout(null); frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); JButton button = new JButton("Click Me"); button.setBounds(100, 50, 120, 40);</pre>

	<pre> JLabel label = new JLabel(); label.setBounds(60, 110, 250, 30); button.addActionListener(new ActionListener() { public void actionPerformed(ActionEvent e) { label.setText("Button Clicked Successfully!"); } }); frame.add(button); frame.add(label); frame.setVisible(true); } } </pre>
Output Screenshots	
Exercise Qns and Code	1. Create a GUI calculator where buttons perform addition and subtraction. <pre> import javax.swing.*.*; public class Exp1 { public static void main(String[] args) { JFrame frame = new JFrame("Calculator"); JTextField num1 = new JTextField(); JTextField num2 = new JTextField(); JButton add = new JButton("Add"); JButton subtract = new JButton("Subtract"); num1.setBounds(50, 40, 200, 30); num2.setBounds(50, 80, 200, 30); add.setBounds(50, 130, 100, 30); subtract.setBounds(160, 130, 100, 30); frame.add(num1); frame.add(num2); frame.add(add); frame.add(subtract); frame.setSize(320, 220); frame.setLayout(null); frame.setVisible(true); add.addActionListener(e -> { int a = Integer.parseInt(num1.getText()); </pre>

```

        int b = Integer.parseInt(num2.getText());
        JOptionPane.showMessageDialog(frame, "Result: " + (a
+ b));
    });
    subtract.addActionListener(e -> {
        int a = Integer.parseInt(num1.getText());
        int b = Integer.parseInt(num2.getText());
        JOptionPane.showMessageDialog(frame, "Result: " + (a
- b));
    });
}
}

```

2. Create a **Bank Balance Calculator GUI application** using Java Swing where the user enters the initial balance and transaction amount. Use buttons to perform **addition (deposit)** and **subtraction (withdrawal)** and display the updated balance.

```

import javax.swing.*;

public class Exp2 {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Bank Balance Calculator");
        JLabel balanceLabel = new JLabel("Initial Balance:");
        JLabel amountLabel = new JLabel("Transaction Amount:");
        JTextField balanceField = new JTextField();
        JTextField amountField = new JTextField();
        JButton deposit = new JButton("Deposit");
        JButton withdraw = new JButton("Withdraw");
        balanceLabel.setBounds(30, 40, 130, 30);
        balanceField.setBounds(160, 40, 180, 30);
        amountLabel.setBounds(30, 90, 130, 30);
        amountField.setBounds(160, 90, 180, 30);
        deposit.setBounds(70, 150, 100, 30);
        withdraw.setBounds(200, 150, 100, 30);
        frame.add(balanceLabel);
        frame.add(balanceField);
        frame.add(amountLabel);
        frame.add(amountField);
        frame.add(deposit);
        frame.add(withdraw);
        frame.setSize(400, 250);
        frame.setLayout(null);
        frame.setVisible(true);
        deposit.addActionListener(e -> {
            double balance =
Double.parseDouble(balanceField.getText());
            double amount =
Double.parseDouble(amountField.getText());
            balance = balance + amount;
            JOptionPane.showMessageDialog(
                frame,
                "Updated Balance: " + balance

```

```

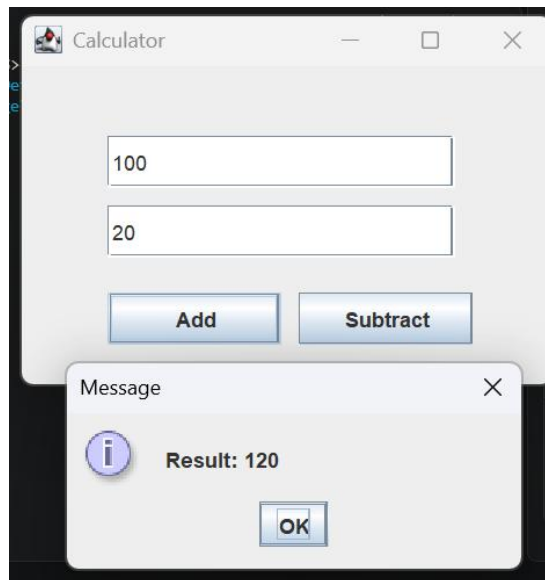
    });
    withdraw.addActionListener(e -> {
        double balance =
Double.parseDouble(balanceField.getText());
        double amount =
Double.parseDouble(amountField.getText());
        balance = balance - amount;
        JOptionPane.showMessageDialog(
            frame,
            "Updated Balance: " + balance
        );
    });
}
}

```

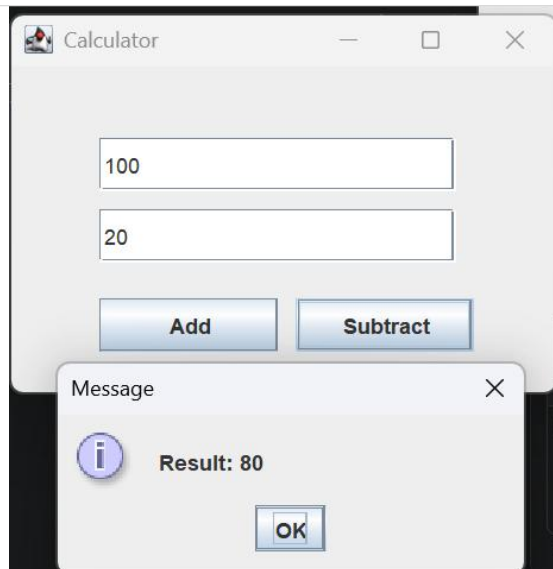
Output Screenshots

Ex1:

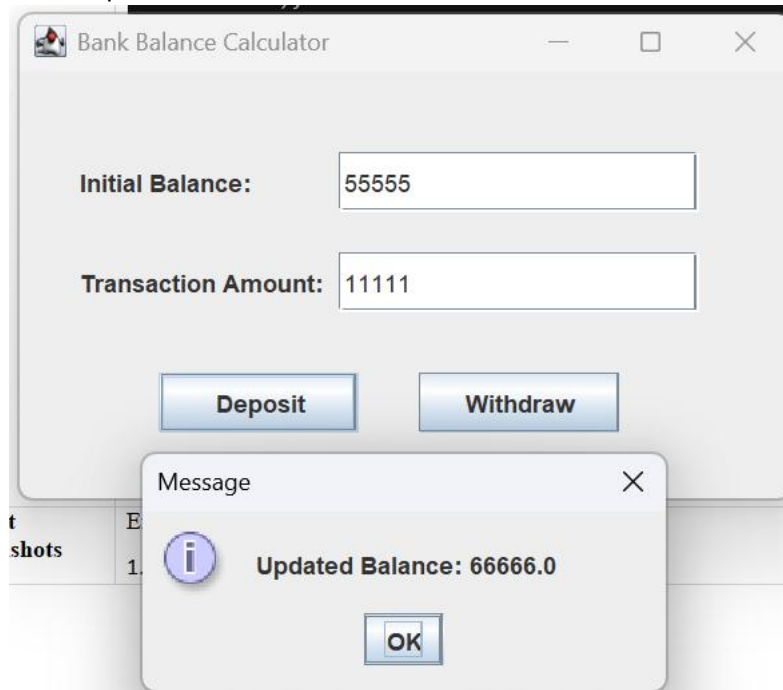
1. Add:



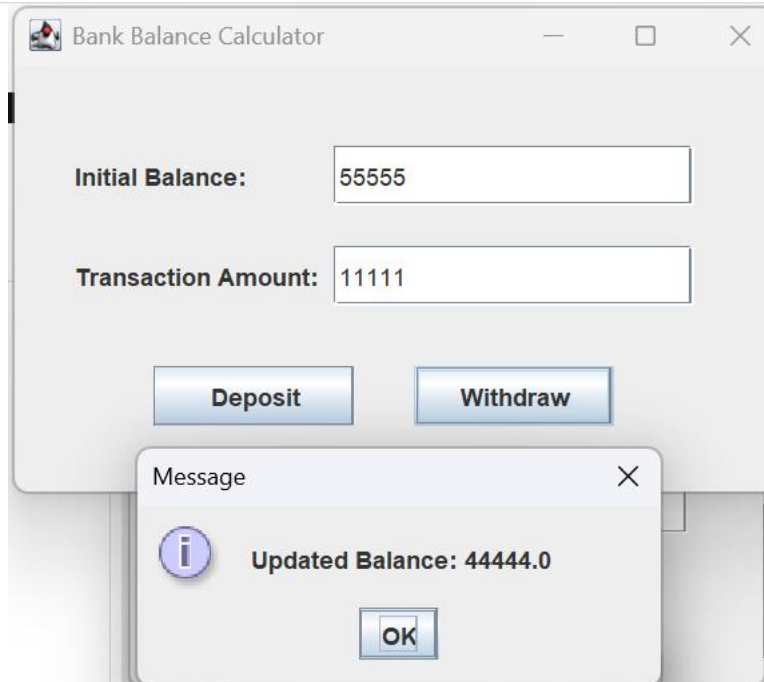
2. Subtract:



Ex2 - Deposit:



Withdrawal:

**Conclusion**

This experiment demonstrates event-driven programming through user interaction listeners. By binding ActionListener implementations and lambda expressions to interactive buttons, it highlights how visual components handle user triggers dynamically to calculate, update, and display application state.